Source housekeeping: what we have and where to find it

The blog, notes, and research papers have different roles

Item	Location	Meaning
Jiaxuan's saved blog	`doc/reading/scientific-scaling-ladder.md`	Secondary synthesis, retained with its original reference links and local figures
Supplied notes	`scaling_ladder_notes.md`	Restructuring and derivations based on the blog; some claims need qualification
Annotated Quarto notes	`doc/reading/deep-dive-notes.qmd`	Generated reading edition with companion links
Source review	`doc/reading/notes-review.qmd`	Checked claims and corrections, with primary evidence
Full source catalogue	`references/SOURCES.md`	Readable names, stable identifiers, PDFs and metadata
Download status	`references/README.md`	Collection scope and unresolved code availability
Machine-readable manifest	`references/download-manifest.json`	Per-reference provenance and retrieval results

The paper folders contain their own `source/` directories for LaTeX. Those are different from the saved blog in `doc/`. An empirical claim should ultimately point to a paper, report or experiment artifact, rather than relying only on the blog or notes.

Collection at a glance

The library contains **84 reference folders**, including **74 PDFs** and **72 successfully collected arXiv source packages**. Per-reference code records show **161 downloaded** and **37 unavailable** archives; the same repository may be associated with several papers.

Topic	Reference folders
Architecture and MoE	6
Data construction, mixtures, and repetition	9
Evaluation and task prediction	6
Marin technical reports	6
Research methodology	2
Model reports: OLMo, Kimi, Nemotron, and others	19
Optimizers and hyperparameter transfer	20
Scaling laws and fitting	14
Training systems and source implementations	2

Counts describe collection availability, not scientific validation. A downloaded source is not automatically a reviewed source, and a paper-linked repository may be evaluation code, data tooling, or a baseline. Current code snapshots are not automatically the revisions used in the reported experiment.

Useful primary sources for this reading project

- Training Compute-Optimal Large Language Models (Chinchilla) — `2203.15556`.

- Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer (μ-Transfer) — `2203.03466v2`.
- Establishing Task Scaling Laws via Compute-Efficient Model Ladders (OLMo Task Ladder) — `2412.04403v2`.
- Power Lines: Scaling Laws for Weight Decay and Batch Size in LLM Pre-training — `2505.13738v2`.
- Predictable Scale: Part I, Step Law — `2503.04715v3`.
- Scaling Data-Constrained Language Models — `2305.16264v5`.
- Olmix: A Framework for Data Mixing Throughout LM Development — `2602.12237`.
- Nemotron 3 Nano: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning — `2512.20848`.
- Nemotron 3 Super: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning — `2604.12374`.
- Nemotron 3 Ultra: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning — `2606.15007`.
- Nemotron-CC: Transforming Common Crawl into a Refined Long-Horizon Pretraining Dataset — `2412.02595`.
- SMELT: Scaling Laws for Compute-Matched MoE Looped Transformers — `2609.01343v3`.

The complete catalogue includes the remaining sources. Use Marin's reading map to locate its implementation examples.

Industry-practice evidence and implementation snapshots

The conversation audit reviews the supplied `scaling_ladder_industry_practice.md` against Step Law v3, Farseer v3, Power Lines v2 and Marin. The data-engineering guide audits actual mixture Parquet records and distinguishes the historical report from newer code. The trainer guide and checkpoint-evaluation guide link the downloaded implementations. The 96-H100 plan is the current budget interpretation.

Pinned trainer archives live in `references/systems/`; ignored checkouts are `tmp/torchtitan`, `tmp/megatron-lm` and `tmp/lm-evaluation-harness`. The source pins and Marin record audit record exactly what was inspected. Records are experiment metadata, not a prepared corpus. The Step Law fitting repository was found through its project site and added to its existing paper folder.

Effort-level reading

The OLMo deep dive connects its corpus work, mixture proxies, Task Ladder and training stages. The Marin deep dive connects Datakit, mixture learning, Delphi and training heuristics. The beginning-to-end design turns those examples into a project plan starting from an available corpus.

Inside a reference folder

```
references/<topic>/<year-readable-title>/
  README.md              description and local entry points
  metadata.json          identifier, URL, version information, download status
  paper.pdf              paper/report, when available
  paper.txt              extracted text for searching
  arxiv-source.tar       original source download, when available
  source/                extracted LaTeX, bibliography and figures
  code/                  discovered repository archives
  article.md             web-first report snapshot, where applicable
  release-recipes/       additional official recipe snapshots, where collected
```

Not every reference has every artifact. Read `metadata.json` for actual availability. Source archives can be gzip-compressed despite the `.tar` suffix. The notes heading map records the section/subsection companion links. The directory map resolves the earlier arXiv-named folders into the readable topic layout.

Focused paper and code replication guides

New focused guides explain Step Law, Farseer, and Power Lines. The MoE architecture comparison uses pinned official Qwen3/OLMoE configs and Marin model code. The Marin optimizer-search manual audits Vizier and multiplier grids; the Datakit code walkthrough traces artifact contracts; the cyber-data extension separates our domain hypothesis from the baseline reproduction.

Routine upkeep

1. When adding evidence, record the primary title, identifier/version, URL and local artifact. Preserve an existing snapshot when a new version could change a numerical claim.
2. Record whether a statement is a reported configuration, measured optimum, mathematical derivation, or our proposed experiment. Add exact section/table/code locators where available.
3. Update the catalogue and status pages from the repository root:

```
python scripts/summarize_references.py
python scripts/build_reference_catalog.py
python scripts/build_source_guide.py
```

4. After editing the supplied Markdown notes, regenerate their Quarto edition and heading map:

```
python scripts/build_notes_quarto.py
```

5. Render the site with `quarto render`, or browse it with `quarto preview`. Generated HTML is in `_site/`; it is ignored by Git. The notes edition and this page are generated source files, so change their generator or the original inputs rather than editing their generated body.

Network collection is separate from housekeeping: the collectors in `scripts/` attempt source/code downloads and record failures. Do not rerun downloads merely to refresh navigation or counts.